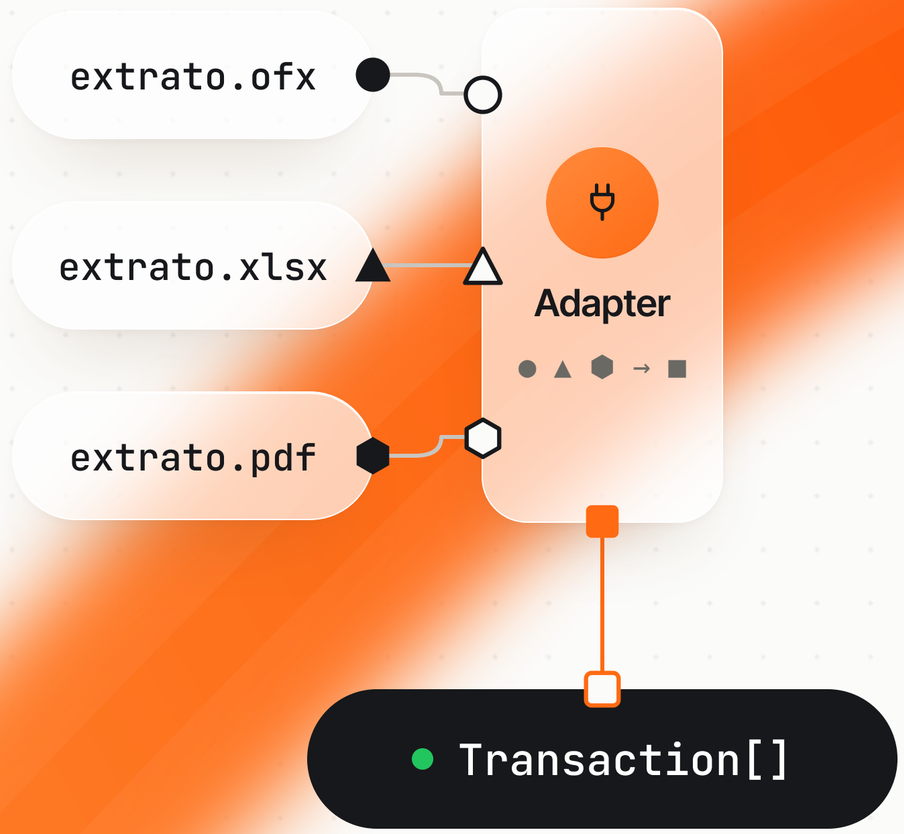
Padrão de projeto estrutural do catálogo GoF

# Adapter

## Um contrato para importar OFX, Excel e PDF sem mexer nas regras do sistema.

Ver o código →

Pedro, Bruno, Ruan e Vitor



O que é?

# Um tradutor entre interfaces

Converte a interface de uma classe em outra interface que o cliente espera. Assim, classes com interfaces incompatíveis passam a trabalhar juntas.

Gamma et al., Padrões de projeto (2000)

Como um **adaptador de tomada**: o aparelho e a parede continuam iguais, só a peça do meio é nova.

Qual a categoria?

### Criação

Como os objetos são criados. Ex.: Singleton, Factory.

### Estrutural Adapter

Como classes e objetos se combinam para formar estruturas maiores. Ex.: Adapter, Facade, Decorator.

### Comportamental

Como os objetos se comunicam. Ex.: Strategy, Observer.

Duas formas: **de objeto** (composição, a que usamos) e **de classe** (herança múltipla).

# A mesma transação chega de três jeitos

O problema: integrar código com interfaces incompatíveis sem alterá-lo. Sem um padrão, o serviço vira um if/else por formato.

● OFX ▲ Excel ⬢ PDF ■ o que o sistema espera

extrato.ofx

**TRNAMT: -150.00** ●

O sinal indica débito ou crédito.

extrato.xlsx

**Valor: 150,00 · Débito** ▲

Cabeçalhos próprios e vírgula decimal.

extrato.pdf

**"… FORNECEDOR -150,00"** ⬢

Texto livre que precisa ser extraído.

Componentes e responsabilidades

# Quatro papéis e como eles interagem

## Client

Usa apenas a interface Target. Não sabe qual formato está por trás.

NO PROJETO
`TransactionImportService`

usa

## Target

Define a interface que o cliente espera receber.

NO PROJETO
`TransactionFileAdapter`

cumpre

## Adapter

Implementa o Target e traduz cada chamada para o Adaptee.

NO PROJETO
`OfxAdapter`, `ExcelAdapter`, `PdfAdapter`

chama

## Adaptee

Código que já existe e é útil, mas tem interface incompatível.

NO PROJETO
`parsers de OFX, planilha e PDF`

**Client** chama parse() → **Adapter** chama o Adaptee → converte o resultado → **Client** recebe Transaction[]

Código: o problema, antes do padrão

# Antes: um if por formato

transaction-import-service.ts — sem Adapter

```ts
class TransactionImportService {
  async import(ext: string, content: Buffer) {
    let transactions: Transaction[];
    if (ext === "ofx") {
      const entries = await parseOfxFile(content);
      transactions = entries.map(/* campos do OFX */);
    } else if (ext === "xlsx") {
      const rows = await parseExcelFile(content);
      transactions = rows.map(/* campos do Excel */);
    } else if (ext === "pdf") {
      transactions = parsePdf(await extractPdfText(content));
    } else throw new Error("Formato não suportado");
    this.validate(transactions); // + regras, persistência
  }
}
```

### ⚠ Formato novo altera o serviço

Mais um else if a cada formato. Viola o princípio Aberto/Fechado.

### ⚠ Leitura e regra de negócio misturadas

A mesma classe sabe ler OFX e validar transações.

### ⚠ Difícil de testar

Não dá para testar o OFX sem passar pelo serviço inteiro.

**Solução: cada tradução vira um adaptador.**

Passos 1 e 2 de 5

# O modelo e o contrato

Primeiro definimos o que o sistema entende. Depois, a regra que todo adaptador precisa cumprir.

A interface não menciona Excel, PDF nem OFX.

1 · transaction.ts

```ts
type TransactionType = "CREDIT" | "DEBIT";

interface Transaction {
  date: string;
  description: string;
  amount: number;
  type: TransactionType;
  externalId?: string;
}
```

2 · transaction-file-adapter.ts — Target

```ts
interface TransactionFileAdapter {
  parse(content: Buffer): Promise<Transaction[]>;
}
```

Passo 3 de 5: um adaptador por formato

# OfxAdapter

ofx.adapter.ts

```ts
class OfxAdapter implements TransactionFileAdapter {
  async parse(content: Buffer): Promise<Transaction[]> {
    const entries = await parseOfxFile(content);
    return entries.map((entry) => ({
      date: normalizeOfxDate(entry.DTPOSTED),
      description: entry.MEMO,
      amount: Math.abs(Number(entry.TRNAMT)),
      type: Number(entry.TRNAMT) < 0 ? "DEBIT" : "CREDIT",
      externalId: entry.FITID,
    }));
  }
}
```

- DTPOSTED → date
- MEMO → description
- TRNAMT → amount
- sinal de TRNAMT → type
- FITID → externalId

Passo 4 de 5

# Um serviço para todos os formatos

Validação, deduplicação e persistência ficam aqui, fora dos adaptadores.

**Recebe a interface, não uma classe concreta.**

Por isso aceita qualquer adaptador.

transaction-import-service.ts

```ts
class TransactionImportService {
  async import(
    content: Buffer,
    adapter: TransactionFileAdapter,
  ): Promise<Transaction[]> {
    const transactions = await adapter.parse(content);
    this.validate(transactions);
    // deduplicação, regras de negócio, persistência

    return transactions;
  }
}
```

Passo 5 de 5

# Passo 5: escolher o adaptador

import-file.ts

```ts
async function importFile(
  ext: string, content: Buffer,
) {
  const adapter = adapters[ext.toLowerCase()];
  if (!adapter) {
    throw new Error("Formato não suportado");
  }
  const service = new TransactionImportService();
  return service.import(content, adapter);
}
```

O objeto **adapters** já faz uma seleção simples, sem precisar de uma Factory completa.

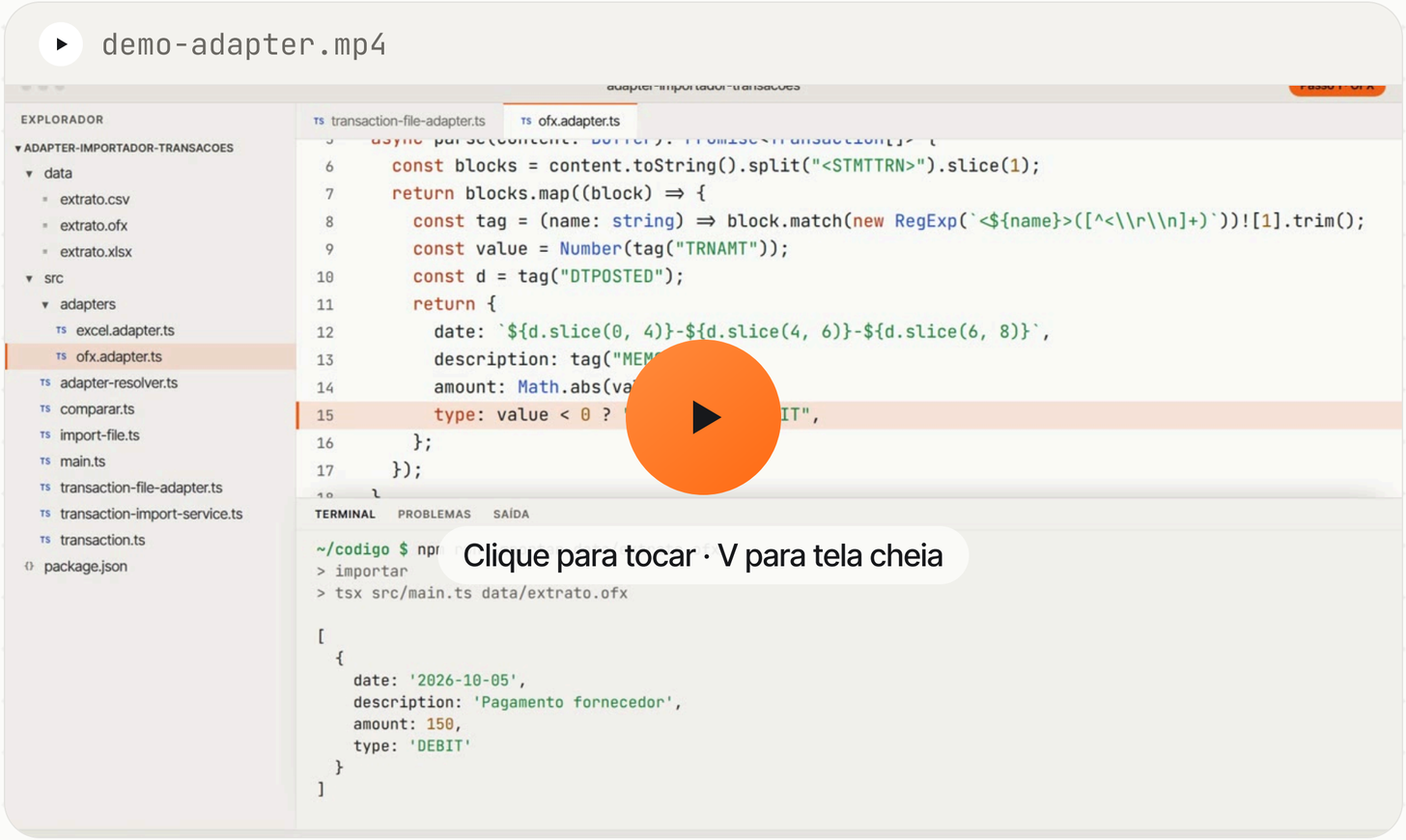
adapter-resolver.ts

```ts
const adapters: Record<string, TransactionFileAdapter> = {
  ofx: new OfxAdapter(),
  xlsx: new ExcelAdapter(),
  pdf: new PdfAdapter(),
};
```

terminal

```
$ importFile("ofx", ofxBuffer)
✓ OfxAdapter → Transaction[]
$ importFile("xlsx", excelBuffer
✓ ExcelAdapter → Transaction[]
$ importFile("csv", csvBuffer)
✗ Error: Formato não suportado
```

# Veja o código rodando




## Roteiro

1. Importar um arquivo OFX
2. A mesma transação em Excel
3. Comparar as duas saídas
4. Adicionar um formato novo

# Relação com outros padrões

## Usados junto com o Adapter

### Factory

Cria ou escolhe o adaptador certo para cada entrada.

No projeto: o mapa adapters, por extensão

### Strategy

Troca um algoritmo por outro em tempo de execução.

No projeto: regras de duplicidade

## Parecidos, mas com outra intenção

### Facade

Cria uma interface **nova e simples** para um subsistema. O Adapter cumpre uma interface **que já existe**.

### Decorator

**Acrescenta comportamento** e mantém a interface. O Adapter **muda a interface**.

### Proxy

Controla o acesso a um objeto com **a mesma interface** dele.

### Bridge

Separa abstração e implementação **desde o projeto**. O Adapter integra o que **já foi feito**.

# Vantagens e desvantagens

## Vantagens

**Responsabilidade única**
A conversão fica separada da regra de negócio.

**Aberto/Fechado**
Formato novo é uma classe nova, sem alterar o serviço.

**Reaproveita o que existe**
Bibliotecas e código legado são usados sem modificação.

**Testes isolados**
Cada adaptador pode ser testado sozinho.

## Desvantagens

**Mais classes e interfaces**
Aumenta a complexidade geral do código.

**Uma camada a mais**
A indireção dificulta entender e depurar o fluxo.

**Pode ser exagero**
Se a classe original pode ser alterada, às vezes é mais simples.

**Conversões pesadas**
Interfaces muito diferentes exigem muito código de tradução.

# Referências


SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2018.
Cap. 7, Projeto e implementação: padrões de projeto

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Padrões de projeto**: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SHVETS, Alexander. **Adapter**. Refactoring.Guru. Disponível em: refactoring.guru/pt-br/design-patterns/adapter

Pergunta para a turma

# Uma fintech vai integrar o Pix de três bancos, cada um com uma API diferente: formato das respostas, códigos de erro e autenticação. Como o padrão Adapter pode ser usado para que o sistema de pagamentos não dependa de nenhum banco específico? E o que a equipe deve evitar colocar dentro dos adaptadores?

Responda no formulário disponibilizado pela professora.